



Session 2:

Build Your First PoC

Session 1 Recap

What You Came In With

AI Coding Landscape

- ✓ Four tool categories and where each belongs
- ✓ Selection framework applied to real scenarios
- ✓ Mental model: AI as the fast junior developer
- ✓ Right tool matched to the right task

Claude Code

- ✓ Terminal interface navigated and understood
- ✓ CLAUDE.md created and verified
- ✓ Permission model — nothing changed without approval
- ✓ Key commands: `/init` · `/compact` · `/cost` · `/clear`

Prompt Engineering

- ✓ Scaffold prompt template applied
- ✓ v1 → v2 improvement loop completed
- ✓ Brief written for a real utility
- ✓ Prompt log started and ready to use

What You Will Build Today



PoC Brief + CLAUDE.md

Scoped, testable,
and ready to build from



Working PoC

Running code that
answers a real
business question



README + Git Commit

Documented, version-
controlled, handoff-ready



Demo-Ready Output

Showable to a
non-technical
stakeholder right now

One session. One real PoC. Yours to keep — and build on in Sessions 3 and 4.

Learning Outcomes

By the end of this session, you will be able to:

- 01 Write a PoC brief that is scoped, testable, and translatable into a CLAUDE.md
- 02 Build a working, runnable PoC from a plain-English brief by applying the five-step build loop
- 03 Use Claude Code to document and add basic error handling to a completed PoC
- 04 Commit a documented PoC to Git using Claude Code-generated commit messages

Today's Agenda

PART 1

~ 20 min

Scoping and Briefing Your PoC

- Brief writing
- CLAUDE.md
- Scope check
- Lab

PART 2

~ 60 min

The Build Loop

- Trainer demo
- Live build Part 1
- Working core

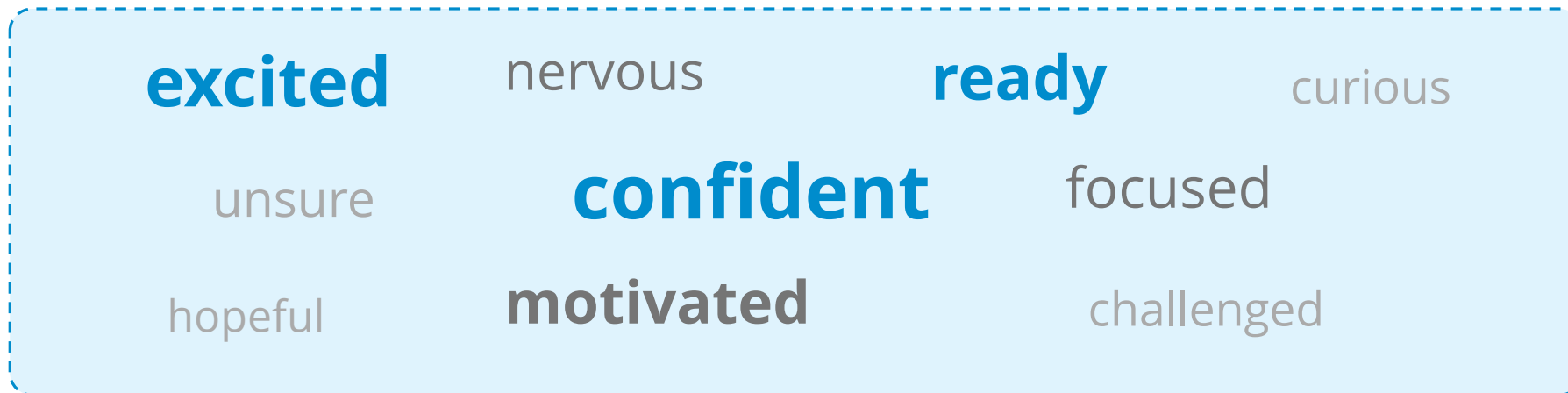
PART 3

~ 40 min

Finish, Document, Stabilise

- README
- Error handling
- Git commit
- Demo share-out

What comes to your mind when you think of building a **new PoC**?



01 SCOPING AND BRIEFING YOUR PoC

What Makes a Good PoC Brief

One user journey

Who uses it and what they do — one path, no branches, no secondary flows

One testable outcome

A clear binary: does this work or not? If you cannot test it, rewrite it.

One integration

One API, one data source, one external connection — not three 'small' ones

Deliberately small scope

If you cannot finish it in a session — cut it until you can. Then cut again.

Questions You Ask Before Building a PoC

The PoC you build should answer something you or your team actually needs to know

“ **Business Outcome**

What business problem am I trying to solve?

“ **User Understanding**

Who is the primary user of this solution?

“ **Scope and Focus**

What is the smallest version of this idea that can demonstrate value?

“ **Data and Inputs**

What information/data does the solution need?

Ask learners to write some similar questions they think before starting on a PoC.

The Four Elements of PoC Brief

Write this before you open Claude Code. Every element has a job.

1 What does it do?

One sentence. Active voice. Name the action and the output.

"A script that reads stale CRM deals and drafts a follow-up email for each one."

3 What does success look like?

One testable outcome. If you cannot verify it in under 60 seconds, rewrite it.

"Correctly identifies deals stale 30+ days and produces a usable email draft for each."

2 Who uses it?

Name the role and the context. Not 'the team' — the specific person in a specific moment.

"A Sales Executive who runs it on Monday mornings before the pipeline review."

4 What is deliberately out of scope?

Name what you will not build. This is as important as what you will.

"No email sending. No CRM write-back. No UI. CLI output only."

Common Scoping Mistakes

✗ Trying to build a product instead of a proof



✓ Ask: what is the one question this needs to answer? Build only that.

✗ Not defining what 'done' looks like before starting



✓ Write your success criterion first. If you cannot test it, rewrite it.

✗ Letting Claude expand scope mid-build



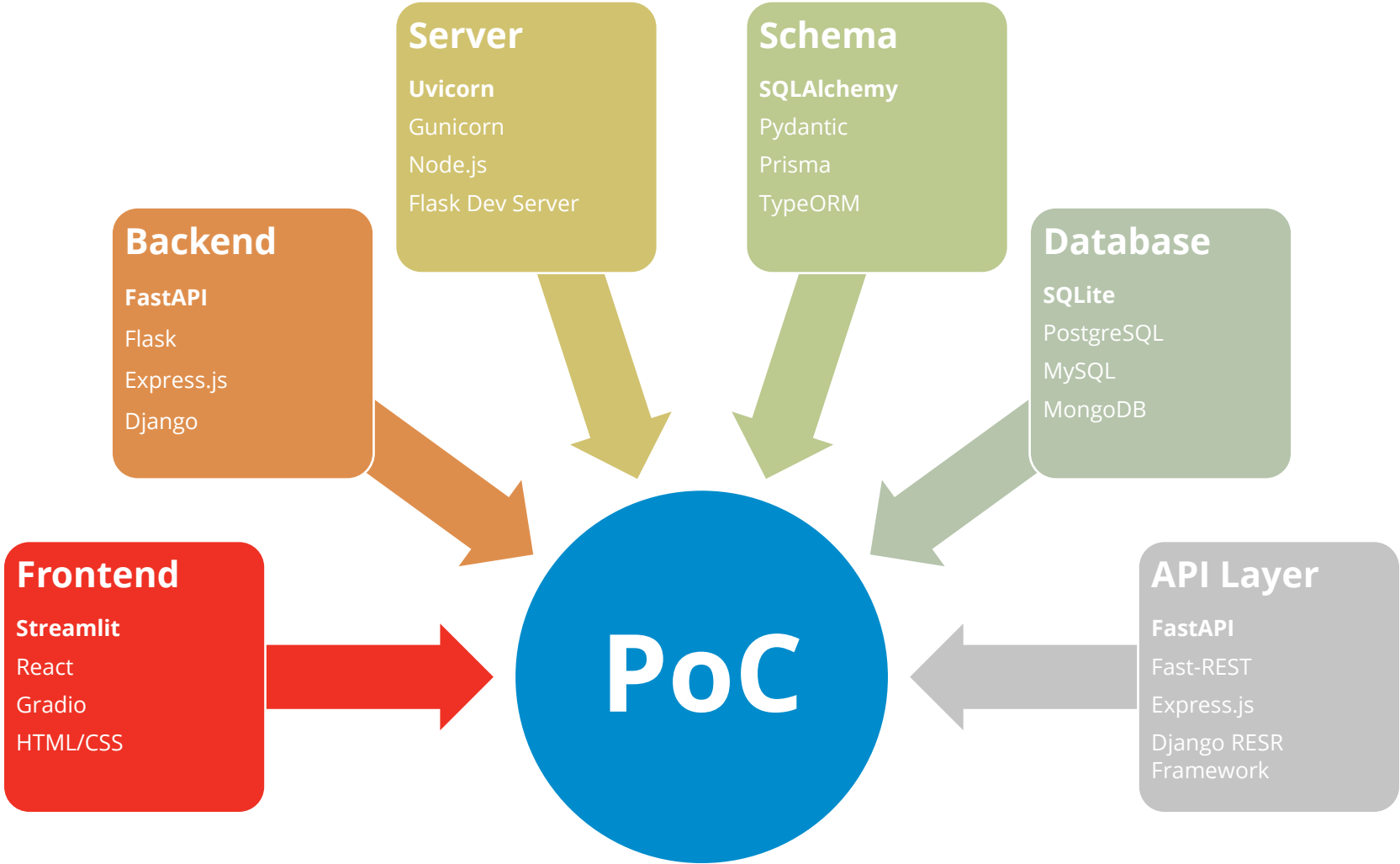
✓ When Claude suggests extras, add them to a backlog — not the current session.

✗ Choosing a toy example instead of a real problem



✓ If you would not use the output in your actual work, pick a different problem.

Components of a PoC



3 Usable Problem Statements

ExpenseFlow

What it does:

- Submit an expense, store it, normalize to INR
- Generate an AI spending insight via Claude API
- Approver accepts or rejects the submission

DischargeFlow

What it does:

- Clinician submits a structured discharge record
- Claude API drafts a plain-language patient summary
- Reviewing clinician approves or rejects the draft

ReturnFlow

What it does:

- Customer raises a return against an order line
- Return window eligibility is computed server-side
- Claude API generates a returns insight. Agent approves or rejects with a disposition

From Brief to CLAUDE.md

Your brief becomes the standing instruction that keeps Claude Code focused throughout the build.

PoC BRIEF

What: Expense submission API that stores expenses, normalises amounts to INR, and generates an AI spending insight per submission

Who: Finance professional submitting and approving expenses – CLI workflow

Success: Expense stored, amount in INR minor units, one insight generated, CLI output shows result

Out of scope: No UI, no email, no Postgres, no multi-currency UI



CLAUDE.md

Project Overview

ExpenseFlow PoC – expense submission, normalisation to INR minor units, and Claude-generated spending insight. Used by finance staff – CLI only.

Tech Stack

Python 3.11 · FastAPI · SQLite
anthropic SDK · Entry: main.py

Constraints

Money as integer minor units (paise).
All amounts normalised to INR on write.
CLI output only. No UI.

Do Not Touch

Do not add a UI, email, or Postgres.
Do not change the money convention.

Is Your PoC Good Enough?

REFLECT

Four questions — one per element of your brief. If any answer is no, revise before you open Claude Code.

1

WHAT DOES IT DO?

Can you describe it in one sentence, active voice, with 'and' used at most once?

✓ **YES:** Good. That is a scoped action with a clear output.

✗ **NO:** Split it. If you used 'and' more than once, you have two PoCs. Pick one.

2

WHO USES IT?

Have you named a specific role in a specific moment — not just 'the team'?

✓ **YES:** Good. You have a real user with a real context.

✗ **NO:** Name the person and the moment. 'A Sales Executive on Monday mornings' is specific. 'The team' is not.

3

WHAT DOES SUCCESS LOOK LIKE?

Can you verify whether it worked in under 60 seconds of running it?

✓ **YES:** Good. You have a testable outcome.

✗ **NO:** Rewrite your success criterion. If you cannot test it in 60 seconds, it is not done.

4

WHAT IS OUT OF SCOPE?

Have you written down at least two things your PoC will not do?

✓ **YES:** Good. That list is what keeps Claude Code from expanding scope mid-build.

✗ **NO:** Write it now. 'No email sending. No UI. No CRM write-back.' Name it before you build it.

Write Your PoC Brief and CLAUDE.md

HANDS-ON LAB

You already chose your problem statement. Now make it buildable.

1

Open your PoC brief template

Use the four-element format: What · Who · Success · Out of scope

2

Write your brief for [PROBLEM STATEMENT]

One sentence per element. Pass the three scope questions before moving on.

3

Run /init to generate a starter CLAUDE.md

Then edit it using your brief as the source of truth.

4

Verify Claude has read it

Ask Claude to name your project purpose and one constraint — without reading any file.

✓ **SUCCESS: Brief written · Scope test passed · CLAUDE.md created · Claude correctly names your project and constraints**

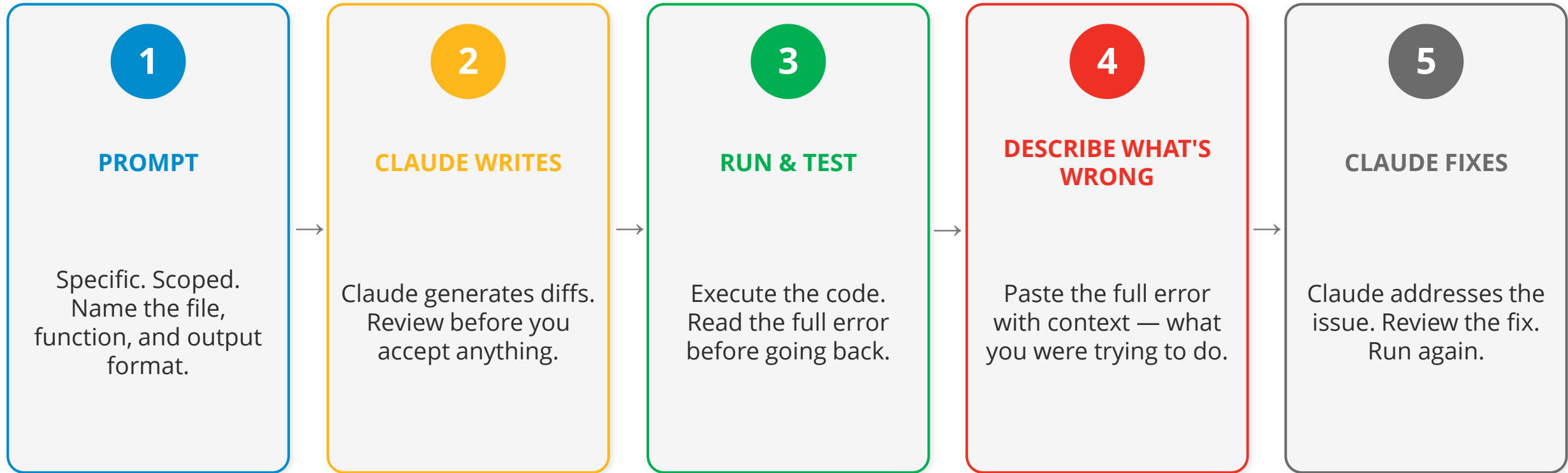
02 THE BUILD LOOP

You have a brief.
Claude Code is open.
Now build.

Everything from this point is the loop — prompt, run, fix, repeat.

The Build Loop — Five Steps

Each Claude response is one step — not a final answer. The loop is the method.



Repeat until the code passes your success criterion — not until it 'feels done'.

Step 1: Prompt — Working With APIs

Most PoCs connect to an API. Claude Code scaffolds the entire call — connection, authentication, and response — without you memorising documentation.

Tell Claude the API name, what you want back, and how errors should be handled. It writes the connection code. You direct the shape — Claude handles the boilerplate.

01 Scaffold the call

Name the API, the endpoint, and what you expect back. Claude writes the connection code.

"Connect to the CRM API and return all accounts with a last_contact field"

02 Handle authentication

Ask Claude to use an environment variable for the API key. Never hardcode credentials.

"Use the API key from the environment variable CRM_API_KEY"

03 Parse the response

Tell Claude which fields you need. It extracts and formats them without you reading the schema.

"Return only: account name, owner, and days since last contact"

Step 2: Claude Writes — Review Before You Accept

Claude does not save anything until you say yes. Your job at this step is to read what it proposes — not to approve it automatically.

A diff is Claude's proposal — a side-by-side view showing exactly what it wants to add, change, or delete. Green means added. Red means removed. Nothing changes on your machine until you press Y to accept.

Did Claude do what I asked?

✓ The new function matches your prompt — correct file, correct output format

✗ Claude added extra logic or features you did not ask for

Reject the extra parts. One thing at a time.

Does anything look out of place?

✓ The change is small and contained — only the lines you expected

✗ Claude touched files or sections you did not mention in your prompt

Ask Claude to explain why it touched those lines before accepting.

Can I explain what changed?

✓ You can describe in one sentence what the diff does and why

✗ You are not sure what changed or why — but it looks fine

If you cannot explain it, do not accept it yet.

Step 3: Run & Test — Reading the Result

You accepted the diff. Now run it. The output tells you what to do next — whether it worked, broke, or produced the wrong result.

Running the code is not optional. Reading the output and deciding what to report back to Claude is the skill that separates fast builds from slow ones.



It worked

The output matches your success criterion. Check it carefully before moving on.

Read the output. If it is correct, move to the next feature. Do not add more before confirming this one works.



It broke with an error

An error message appeared. This is the most useful outcome — it tells you exactly where to look.

Copy the complete error message. Do not trim it. Go to Step 4.



It ran but the result is wrong

No error, but the output does not match what you expected. This is the trickiest case.

Compare actual output to expected output. Write down the difference in plain English. Go to Step 4.

Step 4: Describe What's Wrong — Handle the Error Right

When something breaks, paste the full error into Claude Code with context about what you were trying to do — not just the error message.

Something will break. That is not a problem — it is the process. What you paste back to Claude determines whether the fix works first time or after five more rounds.

✗ What people paste

- ✗ Pasting only the error message — no context
- ✗ Asking Claude to 'fix it' without saying what 'it' is
- ✗ Accepting the first fix without re-running the code
- ✗ Starting a new conversation and losing all context

Claude has no idea where to look or what to fix.



✓ What actually works

- ✓ Paste the full error — never trim it
- ✓ Add one sentence: what you were trying to do
- ✓ Let Claude name the cause before fixing it
- ✓ Run the fix — if it fails again, repeat the loop

Claude now knows exactly where to look and what to fix.

Step 5: Claude Fixes — This Is Not the End

Claude proposes a fix. Your job is the same as Step 2: review before you accept. Then run it again to confirm the fix actually worked.

A fix that makes the error message disappear is not necessarily a fix. It is a hypothesis. Your job is to verify it before moving on.

01

Review the fix first

Before accepting, read what Claude changed. Does the fix address the root cause, or does it just suppress the error message? Ask yourself: could this break something else?

02

Accept and run

Press Y to accept. Run the code immediately. Do not move on before running it. The fix is unverified until the code runs.

03

Check the symptom is gone

The original error or wrong output should no longer appear. If it does, you are back at Step 4. Describe what is still wrong and repeat the loop.

04

Check nothing else broke

Run the full output, not just the fixed part. A fix that solves one thing and silently breaks another is not a fix. Session 3 covers how to write a test that locks this in permanently.

Repeat the loop until the code passes your success criterion. Step 5 is not the end — it leads back to Step 3.

**The loop is running.
The session is stable.
Your core logic produces output.**

There is one question left before the lab:

can someone else actually see what you built?

Next: decide how to make your output showable.

Keeping Claude Code on Track

A single error is handled in one loop pass. A long build session is a different problem — context drifts, responses lose accuracy. These three habits prevent that.

Use Plan Mode

WHEN: Task touches multiple files

Shows Claude's full intention — which files, what changes — before it writes a single line. Review and approve the plan first.

```
$ claude --plan "[your task]"
```

Run `/compact`

WHEN: Responses feel less accurate

Summarises the session and continues with a shorter context. Prevents Claude from forgetting earlier decisions.

```
$ /compact
```

Start a New Conversation

WHEN: Session has drifted past recovery

Clear is faster than salvaging a confused context. Your CLAUDE.md restores project knowledge immediately.

```
$ /clear
```

Making It Demo-able

A working PoC that cannot be shown is a PoC that cannot move forward. Build the output layer early.

CLI Output

Formatted print output that shows what ran and what it produced. Readable without explanation.

Best for: internal demos, technical stakeholders

THE PROMPT:

"Add formatted CLI output showing each result. Use clear labels. Make it readable without explanation."

Simple Front-end

A minimal Streamlit app or HTML page wrapping the core logic. No-code builders work here too.

Best for: client demos, non-technical stakeholders

THE PROMPT:

"Add a minimal Streamlit front-end. Input field for the trigger. Results in a clean table."

Formatted Report

A structured markdown file saved to disk — readable without running anything.

Best for: async review, email attachments

THE PROMPT:

"Output results to report.md. Use headers, bullet points, and a summary section."

Live Build Part 1 — Get a Working Core

HANDS-ON LAB

Follow the same build loop the trainer just ran — applied to your chosen problem statement.

1

Write your first scaffold prompt

Name the file, function, inputs, and outputs. Paste it into Claude Code.

2

Review the diff before accepting

Read every line proposed. Reject anything you did not ask for.

3

Run the code

If it fails — paste the full error back with one sentence of context.

4

Fix and repeat the loop

Work through errors one at a time. Do not move to the next feature until this one runs.

5

Stop when the core works

Main function produces the right output — even if rough. That is the goal for Part 1.

✓ **SUCCESS: Core logic runs · First diff reviewed and accepted · At least one error pasted back with context and fixed**

03 **FINISH, DOCUMENT, STABILISE**

Documentation-First Development

Prompt Claude Code to write the README as you build — not after. A PoC without a README is not handoff-ready.



Forces scope clarity

Writing what it does exposes ambiguity. If you cannot describe it clearly, it is not done.



Makes handoff instant

Anyone picking this up — engineering, a colleague, future you — can start without a briefing.



Confirms the build

Generating the README reveals gaps between what you intended and what Claude actually built.

THE README PROMPT

```
"Write a README.md for this project. Include:  
- What it does (1 paragraph)  
- Who uses it and when  
- How to install and run it  
- What a successful run looks like  
- What is out of scope"
```

Run this now — before you finish building. Edit what is wrong. Keep it current.

Error Handling Basics

The difference between a demo that breaks under questions and one that holds up.

Invalid or missing input

Without: Script crashes with an unreadable stack trace **With:** Clear message: what was expected, what was received

"Add input validation. If [INPUT] is missing or invalid, print a clear error and exit gracefully."

API or service unavailable

Without: Connection timeout with no explanation to the user **With:** Informative failure: what was tried, that it failed, what to check next

"Handle API connection errors. If the request fails, name the endpoint and suggest checking the API key."

Empty or unexpected response

Without: KeyError or AttributeError mid-execution **With:** Handled: logs the empty response, continues or exits cleanly — no crash

"Handle empty responses from [API]. If no results, print a message and exit — do not crash."

Your PoC is done when it answers the question it was built to answer.

✗ Not when it handles every edge case

✗ Not when it is polished or production-ready

✗ Not when you have simply run out of time

Go back to your success criterion. If it passes — you are done.

The Self-Review Prompt

Before you call it done — ask Claude to stress-test it. 30 seconds. Saves your demo.

THE PROMPT

```
"What are the top 3 things most likely to break or fail in a live demo of this PoC? Be specific."
```

Classify each risk Claude identifies:

Blocking

Breaks the demo entirely

→ Fix it now — before the commit

Deferrable

Edge case unlikely in a controlled demo

→ Note it in the README. Move on.

Known Limitation

Out of scope by design

→ Confirm it is in your 'out of scope' list.

Saving Your Work — Git Commit

Commit before Session 3. Claude generates the message. You review and push.

Generate the commit message

1

```
$ claude "Write a Git commit message for this session. Include what it does, what it does not do, and any known limitations."
```

Review it. Edit if vague.

Stage your files

2

```
$ git add .
```

Check git status first. Only stage files you intended to change.

Commit with the generated message

3

```
$ git commit -m "[CLAUDE-GENERATED MESSAGE]"
```

This PoC runs through Sessions 3 and 4. A clear message matters.

Live Build Part 2 — Complete, Document, Commit

HANDS-ON LAB

Finish the PoC. Make it explainable. Make it committable.

1

Complete the working core

If Part 1 left anything unfinished — finish it. One function, one clean output.

2

Run the self-review prompt

Fix anything blocking. Note what is deferrable in the README.

```
$ claude "Top 3 things most likely to fail in a live demo?"
```

3

Generate and review your README

Run the README prompt. Fix any inaccuracies before saving the file.

```
$ claude "Write a README.md for this project..."
```

4

Add basic error handling

Apply the three error handling prompts to your most likely failure points.

5

Commit to Git

Generate commit message · Stage files · Commit · You are done for Session 2.

```
$ git add . && git commit -m "[message]"
```

✓ **SUCCESS: PoC runs · Self-review done · README saved · Error handling added · Committed to Git**

60-Second Demo

SHARE-OUT

One or two volunteers. 60 seconds each. Show it running — not on slides.

COVER THREE THINGS IN 60 SECONDS

1

What question does your PoC answer?

One sentence. The problem it was built to solve.

2

Show it running.

Execute it live. Real output. Not a screenshot or a description.

3

What would you change with another 30 minutes?

The most important gap you are aware of.

Everyone else: note one thing you would borrow from what you just saw.

What You Now Have

SCOPED & BRIEFED

- ✓ PoC brief: one journey, one integration, one testable outcome
- ✓ Scope test passed
- ✓ CLAUDE.md created and verified

BUILT & RUNNING

- ✓ Working PoC — core logic runs
- ✓ Build loop applied: prompt → run → fix → repeat
- ✓ Self-review prompt run — blocking risks addressed

DOCUMENTED & COMMITTED

- ✓ README generated and reviewed
- ✓ Basic error handling added
- ✓ Committed to Git with a descriptive message

This PoC continues in Sessions 3 and 4. Keep it open — you will build on it.

What's Next — Session 3

Debug, Review & Improve Your Code · Same PoC. New layer.

AI-Assisted Debugging

01

Apply the 5-step debug framework. Read stack traces with Claude as co-pilot. Find and fix the failures before they reach a stakeholder.

AI-Assisted Code Review

02

Run a structured review prompt on your own PoC. Identify severity-ranked issues. Apply the fixes. Re-run the review to confirm.

Development Workflows

03

Commit clean work to Git with Claude-generated messages. Produce a one-page technical overview your engineering team can act on.

Bring: your committed PoC · your Git branch · your prompt log from today.

Q&A



What is the correct **ORDER** for adding a feature to an existing telecom billing service?

- A. /plan → implement → /init → /diff
- ☒ B. /init + /memory → /plan → implement → /diff → /code-review
- C. implement → /code-review → /init → /plan
- D. /clear → implement → /security-review → /init

ANSWER

B Load context first, plan, implement, then inspect and review. Understanding precedes editing.

You must estimate effort on an unfamiliar EHR-integration repo in two days. Which approach gives the best-grounded estimate?

- A** A. Ask Claude to guess story points from the README alone
- B** B. Run `/code-review --fix` on everything and count the fixes
- ✓ C** C. `/init`, ask architecture questions, map modules and complexity, list unknowns, then size by component
- D** D. Immediately `/plan` the new feature and use the plan length as the estimate

ANSWER

C Understanding before sizing. Estimate by component with explicit unknowns; the PM owns the final number.

Before relying on an external tool (database, Jira) inside a session, which steps are appropriate? Select all that apply.

- ☒ **A. /mcp to set up or verify the server connection**
- ☒ **B. /permissions to set the approval rules you want**
- ☐ **C. /usage to check token spend**
- ☒ **D. Confirm the connector resolves to the right project or board**

ANSWER

A, B, D /mcp registers and verifies servers, /permissions sets trust boundaries, and you confirm it points to the right data. /usage is about spend, not tool setup.

The key difference when using Claude on front-end versus back-end work is:

- ☐ A. Claude can only do back-end
- ☒ B. Back-end is verified by running tests; front-end usually needs you to check the rendered UI, because Claude cannot see pixels
- ☐ C. Front-end needs no verification at all
- ☐ D. There is no practical difference

ANS

Two different verification loops. Claude reasons over code, not over pixels, so you close the loop on anything visual in a real browser.

A non-technical analyst wants to build a small tool with Claude. Best approach?

- ☐ A. Tell them to learn to code first
- ☒ B. Start from a plain-English brief and the build loop; Claude writes and explains, they verify the behaviour and the outcome
- ☐ C. Conclude they cannot use Claude
- ☐ D. Have them paste unknown code from the internet and hope

Automation · Augmentation

Claude augments the non-coder. They own the problem and the verification; Claude owns the syntax. Outcome over implementation.

Project estimation. You are a Program Manager with two days to estimate enhancements to a hospital DischargeFlow service. You have read access to the repo and Claude Code. Mark each correct step.

RIGHT

Run /init to generate CLAUDE.md and get a map of the stack, modules, and data stores.

RIGHT

Ask Claude architecture questions (@src) about request flow, dependencies, and where discharge logic lives.

RIGHT

Run /code-review --fix across the whole repo and base the estimate on the number of auto-fixes.

RIGHT

Have Claude flag unknowns and external integrations (HL7, FHIR) that widen the estimate

RIGHT

Size the work by component with optimistic, likely, and pessimistic ranges, and list assumptions.

RIGHT

Skip understanding and ask Claude to 'just give a number in story points' from the README

RIGHT

Treat Claude's number as final and commit the date to the client without human review.

Project estimation. You are a Program Manager with two days to estimate enhancements to a hospital DischargeFlow service. You have read access to the repo and Claude Code. Mark each correct step.

RIGHT

Run /init to generate CLAUDE.md and get a map of the stack, modules, and data stores. *A grounded map is the right starting point.*

RIGHT

Ask Claude architecture questions (@src) about request flow, dependencies, and where discharge logic lives. *Understanding drives a defensible estimate.*

WRONG

Run /code-review --fix across the whole repo and base the estimate on the number of auto-fixes. *Review is not sizing, and --fix mutates code you only have read access to assess.*

RIGHT

Have Claude flag unknowns and external integrations (HL7, FHIR) that widen the estimate. *Surfacing unknowns is exactly what a two-day estimate needs.*

RIGHT

Size the work by component with optimistic, likely, and pessimistic ranges, and list assumptions. *Ranged, assumption-based sizing is sound estimation practice.*

WRONG

Skip understanding and ask Claude to 'just give a number in story points' from the README. *An ungrounded number is a guess, not an estimate.*

WRONG

Treat Claude's number as final and commit the date to the client without human review. *The PM owns the estimate. Always verify before committing to a client.*

LESSON Understand and bound the work first; the PM owns the final number.

Testing a new feature. You are a QA Lead. Engineering shipped a 'partial refund' feature in the ReturnFlow checkout service. You have Claude Code. Mark each step.

- RIGHT** Read CLAUDE.md (or /init) to understand the service, then @ the new feature files to see what changed.
- RIGHT** Ask Claude to derive test cases from the acceptance criteria, including edge cases (zero amount, over-refund, currency rounding).
- RIGHT** Ask Claude to write code that breaks the live payment gateway to 'prove' it is fragile. .
- RIGHT** Have Claude generate unit and integration tests, then you run them and review the failures.
- RIGHT** Use /security-review on the new payment path.
- RIGHT** Accept 'all tests pass' from Claude without running the suite yourself.
- RIGHT** Ask Claude to map the feature's regression surface on existing flows before sign-off.

Testing a new feature. You are a QA Lead. Engineering shipped a 'partial refund' feature in the ReturnFlow checkout service. You have Claude Code. Mark each step.

RIGHT

Read CLAUDE.md (or /init) to understand the service, then @ the new feature files to see what changed. *Know the change surface before writing tests.*

RIGHT

Ask Claude to derive test cases from the acceptance criteria, including edge cases (zero amount, over-refund, currency rounding). *Edge-case coverage is the point of the exercise.*

WRONG

Ask Claude to write code that breaks the live payment gateway to 'prove' it is fragile. *That is attacking a production system. Claude will not help, and it is not testing.*

RIGHT

Have Claude generate unit and integration tests, then you run them and review the failures. *Generate fast, but you run and judge the results.*

RIGHT

Use /security-review on the new payment path. *Payment paths warrant a read-only security pass.*

WRONG

Accept 'all tests pass' from Claude without running the suite yourself. *Never sign off on a claimed result you did not execute.*

RIGHT

Ask Claude to map the feature's regression surface on existing flows before sign-off. *Regression impact is core QA diligence.*

LESSON Generate tests fast, but you run them and own the sign-off.

Thank You.



A strategic partner to the most admired Fortune 500® companies globally, we help power every human decision in the enterprise by bringing advanced analytics & AI, engineering and design.



www.fractal.ai